

GuardApp

Android Environment Detection SDK

Documentação Técnica com Contexto Histórico

Projeto: Incognia Technical Challenge

Propósito: Detecção de ambientes adulterados em Android

Linguagem: Kotlin / Android SDK

Cobertura: Emulador, Root, Clone App, Hooking, Reempacotamento

Cada classe, função e decisão de design é explicada junto
com a história que motivou sua existência.

Sumário

1	Visão Geral da Arquitetura	3
1.1	Estrutura de Pacotes	3
1.2	Fluxo de Execução	3
2	EnvironmentAnalyzer.kt — O Contrato	4
2.1	O que e	4
2.2	Por que começar aqui	4
2.3	Decisoes de Design	4
3	DetectionSignal.kt — Os Dados	5
3.1	Os tres tipos	5
4	SecurityReport.kt — O Resultado	7
4.1	isTampered	7
4.2	riskScore	7
4.3	emulatorProbability — A Matematica Bayesiana	7
5	EnvironmentGuard.kt — O Orquestrador	9
5.1	private constructor + companion object	9
5.2	Execucao Paralela com Coroutines	9
5.3	runCatching — Isolamento de Falhas	10
6	EmulatorAnalyzer.kt — Detectando Emuladores	11
6.1	Camada 1: checkBuildProperties()	11
6.2	Camada 2: checkSystemProperties() via Reflection	11
6.3	Camada 3: checkEmulatorFiles()	12
6.4	Camada 4: checkCpuInfo() com charArrayOf	12
7	CloneAnalyzer.kt — Detectando Apps Clone	13
7.1	Camada 3: UID Offset — A Deteccao Mais Elegante	13
7.2	Camada 5: /proc/self/maps e .so baked-in	14
7.3	Injecao de Dependencias para Testabilidade	14
8	VirtualizationAnalyzer.kt — Detectando Hooking	16
8.1	checkHookFiles() — Artefatos de Cada Framework	16
8.2	checkXposedClassLoader() — O Classpath como Detector	16
8.3	checkFrida() — A Deteccao Mais Sofisticada	17
8.3.1	O formato do /proc/net/tcp	17
9	RootDetector.kt — Detectando Root	19
9.1	Camada 1: checkSuBinaries()	19
9.2	Camada 3: checkDangerousProps()	19
9.3	Camada 4: checkWritableSystemPaths()	20
10	SignatureAnalyzer.kt — Verificando Integridade do APK	21
10.1	Camada 1: Debug Key Detection	21
10.2	Camada 2: SHA-256 Hash Comparison	21

10.3 A API Dividida: Android P (2018)	22
11 DetectionSignatures.kt — O Catalogo Centralizado	23
12 AndroidManifest.xml — A Configuracao	24
13 Conceitos Transversais	25
13.1 O Padrao de Injecao de Dependencias	25
13.2 runCatching e Result<T>	25
13.3 O Binder IPC do Android	25
14 Bug Encontrado e Corrigido	26
14.1 Como o bug foi encontrado	26
14.2 Por que os testes nao pegaram	26
14.3 Impacto	26
15 Resumo: Decadas de Historia em 500 Linhas de Kotlin	27

Visão Geral da Arquitetura

O GuardApp é um SDK Android que detecta ambientes adulterados em tempo de execução. Responde a uma pergunta crítica para aplicações financeiras e de segurança:

“Este app está rodando num dispositivo legítimo, ou em ambiente controlado por um atacante?”

Estrutura de Pacotes

```

1  com.incognia.guardapp
2      detection/
3          EnvironmentAnalyzer.kt      # contrato (interface)
4          DetectionSignal.kt          # tipos de dados
5          SecurityReport.kt           # resultado final
6          EnvironmentGuard.kt         # orquestrador (facade)
7          DetectionSignatures.kt      # catalogo de assinaturas
8      analyzers/
9          EmulatorAnalyzer.kt         # detecta AVD/QEMU/Genymotion
10         CloneAnalyzer.kt            # detecta VirtualApp/Parallel
11                                     Space
12         VirtualizationAnalyzer.kt   # detecta Frida/Xposed
13         RootDetector.kt             # detecta root/su/Magisk
14         SignatureAnalyzer.kt        # verifica assinatura do APK
15  ui/
16      MainActivity.kt                # interface do usuario

```

Fluxo de Execução

```

1  MainActivity
2      |
3      v
4  EnvironmentGuard.generateReport()
5      |
6      +-- async --> EmulatorAnalyzer.analyze()      ---+
7      +-- async --> CloneAnalyzer.analyze()          ---+
8      +-- async --> VirtualizationAnalyzer.analyze() ---+-->
9      flatten()
10     +-- async --> RootDetector.analyze()            ---+      |
11     +-- async --> SignatureAnalyzer.analyze()        ---+      v
12                                                         SecurityReport
13                                                         |
14                                                         isTampered
15                                                         /
16                                                         riskScore
17                                                         emulatorProbability
18                                                         summary

```

Os quatro analisadores rodam em paralelo via coroutines Kotlin. Qualquer falha individual é capturada silenciosamente — o relatório continua com os sinais dos demais analisadores.

EnvironmentAnalyzer.kt — O Contrato

```
1 fun interface EnvironmentAnalyzer {  
2     fun analyze(context: Context): List<DetectionSignal>  
3 }
```

O que é

Uma `fun interface` — um contrato com um unico metodo. Qualquer classe que quiser ser um analisador de ambiente implementa esse contrato.

Por que começar aqui

Toda a arquitetura do projeto nasce desse contrato. Antes de falar de emulador, root ou Frida, a pergunta que o projeto responde é: *“qual é a forma padrao de perguntar ao ambiente se algo está errado?”*

Historia: fun interface e SAM Conversion

O Java introduziu interfaces funcionais no Java 8 (2014) com a anotacao `@FunctionalInterface`. A ideia veio das linguagens funcionais — Haskell, ML, Scala — onde funcoes são valores de primeira classe.

O Kotlin evoluiu o conceito com `fun interface` (SAM — Single Abstract Method): qualquer interface com um unico metodo pode ser instanciada com uma lambda diretamente, sem precisar de classe anonima.

```
1 // Sem fun interface (verbose):  
2 val fake = object : EnvironmentAnalyzer {  
3     override fun analyze(ctx: Context) = listOf(sinal)  
4 }  
5  
6 // Com fun interface (SAM):  
7 val fake = EnvironmentAnalyzer { listOf(sinal) }
```

Nos testes, isso é fundamental — criar analisadores falsos fica trivial.

Decisoes de Design

- Recebe `Context` — passaporte do Android para acessar `PackageManager`, `ApplicationInfo`, etc.
- Retorna `List<DetectionSignal>` e não `Boolean` — um analisador pode encontrar varios problemas simultaneamente.
- É uma `interface` e não `abstract class` — permite composicao sem heranca rigida.

DetectionSignal.kt — Os Dados

```
1  enum class SignalCategory(val displayName: String) {
2      EMULATOR("Emulator"),
3      CLONE_APP("Clone / Dual-Space App"),
4      VIRTUALIZATION("Virtualization / Hooking"),
5      ROOT("Root / Superuser"),
6      SIGNATURE("Signature / Integrity"),
7      SUSPICIOUS_PATH("Suspicious Path"),
8      SYSTEM_PROPERTY("Suspicious System Property"),
9  }
10
11  enum class Severity { LOW, MEDIUM, HIGH }
12
13  data class DetectionSignal(
14      val category: SignalCategory,
15      val description: String,
16      val severity: Severity,
17      val explanation: String = "",
18  )
```

Os tres tipos

SignalCategory — de onde veio o sinal (qual analisador, qual ameaca).

Severity — o peso do sinal:

Valor	Significado	Contribuicao ao riskScore
LOW	Informacional, nao prova nada sozinho	0
MEDIUM	Suspeito, aumenta o risco	1
HIGH	Forte indicador de adulteracao	3

DetectionSignal — o sinal em si com quatro campos.

Historia: data class no Kotlin

Em Java, criar uma classe de dados exigia escrever manualmente `equals()`, `hashCode()`, `toString()` e `copy()` — dezenas de linhas de codigo boilerplate propensas a bugs.

A JetBrains criou o Kotlin em 2011 (lancado em 2016) com `data class`: uma unica palavra-chave gera todos esses metodos automaticamente.

```
1  // Java: ~40 linhas
2  // Kotlin:
3  data class DetectionSignal(
4      val category: SignalCategory,
5      val description: String,
6      val severity: Severity,
7  )
```

Importante para os testes: `assertEquals(sinal1, sinal2)` funciona por valor

(compara campos), nao por referencia (compara ponteiros).

SecurityReport.kt — O Resultado

```
1  data class SecurityReport(val signals: List<DetectionSignal>) {
2
3      val isTampered: Boolean
4          get() = signals.any { it.severity != Severity.LOW }
5
6      val riskScore: Int
7          get() = signals.sumOf { signal ->
8              when (signal.severity) {
9                  Severity.HIGH    -> 3
10                 Severity.MEDIUM -> 1
11                 Severity.LOW     -> 0
12             }.toInt()
13      }
14
15      val emulatorProbability: Float
16          get() {
17              var p = 0.0
18              emulatorSignals.forEach { signal ->
19                  p += (1.0 - p) * when (signal.severity) {
20                      Severity.HIGH    -> 0.80
21                      Severity.MEDIUM -> 0.50
22                      Severity.LOW     -> 0.10
23                  }
24              }
25              return p.toFloat().coerceIn(0f, 1f)
26          }
27  }
```

isTampered

Se existe qualquer sinal MEDIUM ou HIGH, o ambiente é considerado adulterado. LOW sozinho é apenas informativo.

riskScore

HIGH vale 3 pontos, MEDIUM vale 1. Permite comparar gravidade entre dispositivos. Um score 9 é muito mais grave que score 3, mesmo que ambos sejam `isTampered = true`.

emulatorProbability — A Matemática Bayesiana

Historia: Thomas Bayes e Probabilidade Independente (1763)

Thomas Bayes, matemático inglês, publicou em 1763 (postumamente) o teorema que leva seu nome. A ideia central: **evidências independentes se acumulam**.

A formula usada no codigo:

$$P = 1 - \prod_i (1 - p_i)$$

Na pratica: cada novo sinal contribui com a probabilidade restante de ainda nao ter sido detectado:

Sinal	Calculo	Resultado
1 HIGH (0.80)	$0 + (1 - 0) \times 0.80$	0.80
2 HIGH (0.80)	$0.80 + (1 - 0.80) \times 0.80$	0.96
3 MEDIUM (0.50)	$0.96 + (1 - 0.96) \times 0.50$	0.98

Por que nao somar? Porque $0.80 + 0.80 + 0.50 = 2.10$ e impossivel (probabilidade nao pode ultrapassar 1.0).

EnvironmentGuard.kt — O Orquestrador

```
1 class EnvironmentGuard private constructor(private val context:
   Context) {
2
3     private val analyzers: List<EnvironmentAnalyzer> = listOf(
4         EmulatorAnalyzer(),
5         CloneAnalyzer(),
6         VirtualizationAnalyzer(),
7         SignatureAnalyzer(),
8         RootDetector(),
9     )
10
11     suspend fun generateReport(): SecurityReport =
12         coroutineScope {
13             val deferred = analyzers.map { analyzer ->
14                 async(Dispatchers.Default) {
15                     runCatching { analyzer.analyze(context)
16                     }.getOrElse(emptyList())
17                 }
18             }
19             SecurityReport(deferred.awaitAll().flatten())
20         }
21
22     companion object {
23         fun create(context: Context): EnvironmentGuard =
24             EnvironmentGuard(context.applicationContext)
25     }
26 }
```

private constructor + companion object

O construtor é privado — ninguém cria `EnvironmentGuard(context)` diretamente. A única forma é via `EnvironmentGuard.create(this)`.

Isso força o uso de `context.applicationContext` em vez do `Activity context`, evitando memory leak — a `Activity` pode ser destruída, o `applicationContext` não.

Execução Paralela com Coroutines

Historia: Coroutines e Concorrência

O conceito de coroutines foi proposto por **Melvin Conway** em 1963 — o mesmo Conway da Lei de Conway. A ideia: funções que podem suspender e resumir execução sem bloquear uma thread.

O Kotlin implementou coroutines como biblioteca de primeira classe em 2018. `async/await` permite paralelismo sem callbacks ou thread pools manuais:

```
1 // Sem paralelismo (sequencial - mais lento):
2 analyzers.forEach { it.analyze(context) }
```

```
3
4 // Com async/await (paralelo):
5 val deferred = analyzers.map { async(Dispatchers.Default) {
6     it.analyze(context) } }
7 deferred.awaitAll() // espera todos terminarem
```

Cada analisador roda em `Dispatchers.Default` — um pool de threads otimizado para operacoes de CPU. File I/O e network probes nao bloqueiam uns aos outros.

runCatching — Isolamento de Falhas

```
1 runCatching { analyzer.analyze(context)
2     }.getOrElse(emptyList())
```

Se um analisador crasha, o erro e capturado silenciosamente e retorna lista vazia. Um analisador quebrado nao derruba o relatorio inteiro.

EmulatorAnalyzer.kt — Detectando Emuladores

```
1 override fun analyze(context: Context): List<DetectionSignal> =
2     checkBuildProperties() +
3     checkSystemProperties() +
4     checkEmulatorFiles() +
5     checkCpuInfo()
```

Historia: QEMU e o AVD (2003-2007)

Fabrice Bellard, engenheiro frances, criou o **QEMU** em 2003 — emulador de processador completo que executa instrucoes ARM num computador x86. Bellard e famoso na comunidade por projetos extremos: tambem criou o FFmpeg.

O Google pegou o QEMU, colocou o Android por cima, e chamou de **AVD** (Android Virtual Device) em 2007. Os engenheiros escolheram “goldfish” como nome do perfil de hardware virtual. Ninguem pensou que isso seria um ponto fraco de seguranca duas decadas depois.

Camada 1: checkBuildProperties()

```
1 val hardware = Build.HARDWARE.lowercase()
2 if (hardware in setOf("goldfish", "ranchu", "vbox86", "nox"))
3     signal("Emulator hardware detected: ${Build.HARDWARE}")
```

`Build.*` sao constantes gravadas em `/system/build.prop` pelo fabricante durante a compilacao. Em emuladores, a Google nao se preocupa em disfarcar:

Campo	Emulador vs Device Real
<code>Build.HARDWARE</code>	goldfish vs shiba (Pixel 8 Pro)
<code>Build.FINGERPRINT</code>	comeca com generic vs hash do fabricante
<code>Build.MODEL</code>	Android SDK built for x86 vs Pixel 8 Pro
<code>Build.BRAND</code>	generic vs google

Camada 2: checkSystemProperties() via Reflection

```
1 val clazz = Class.forName("android.os.SystemProperties")
2 val get = clazz.getMethod("get", String::class.java,
3     String::class.java)
4 val value = get.invoke(null, "ro.kernel.qemu", "")
```

Historia: Reflection no Java (1997)

A Sun Microsystems adicionou Reflection ao Java 1.1 em 1997 como parte do Java Core Reflection API. A ideia: inspecionar e invocar qualquer classe e metodo em runtime, ignorando visibilidade (`private`, `internal`).

`android.os.SystemProperties` e uma API **interna** do Android — nao esta no

SDK publico, nao compila se voce chamar diretamente. Via Reflection, voce burla essa restricao em runtime.

Camada 3: checkEmulatorFiles()

```
1 val EMULATOR_PATHS = listOf(  
2     "/dev/socket/qemud",           // socket QEMU host<->guest  
3     "/dev/qemu_pipe",              // pipe de alta performance  
4     "/dev/socket/genyd",           // Genymotion  
5     "/system/bin/nox-prop",        // Nox  
6 )  
7 EMULATOR_PATHS.filter { File(it).exists() }
```

Arquivos de dispositivo criados pelo kernel do emulador no boot. Em hardware fisico, simplesmente nao existem.

Camada 4: checkCpuInfo() com charArrayOf

```
1 // "goldfish" nunca aparece como literal no APK compilado:  
2 val goldfish =  
3     charArrayOf('g','o','l','d','f','i','s','h').concatToString()  
4 val cpuInfo = File("/proc/cpuinfo").readText()  
5 if (cpuInfo.contains(goldfish)) { ... }
```

Historia: /proc/cpuinfo e a tecnica charArrayOf

Tom Killian criou o /proc no Bell Labs em 1984. O arquivo /proc/cpuinfo expoe o tipo de CPU do hardware diretamente do kernel. Em emuladores QEMU, aparece “QEMU Virtual CPU” e “goldfish” — o kernel do emulador se autoidentifica.

A tecnica charArrayOf vem da pratica de obfuscao de malware: se voce escrever "goldfish" como string literal, ela aparece em texto puro no APK e qualquer um executa `strings apk.apk | grep goldfish`. Construindo em runtime a partir de chars individuais, a string nunca existe como literal no binario.

CloneAnalyzer.kt — Detectando Apps Clone

```
1 class CloneAnalyzer(  
2     private val uidProvider: () -> Int = { Process.myUid() },  
3     private val fileProbe: (String) -> Boolean = {  
4         File(it).exists() },  
5     private val packageChecker: (Context, String) -> Boolean =  
6         { ctx, pkg ->  
7             runCatching { ctx.packageManager.getPackageInfo(pkg,  
8                 0); true }  
9             .getOrDefault(false)  
10        },  
11     private val dataDirProvider: (Context) -> String? = {  
12         it.applicationInfo?.dataDir },  
13 ) : EnvironmentAnalyzer
```

Historia: VirtualApp e o mercado de clones (2015)

Em 2015, um estudante universitario chinees conhecido como **Lody** (GitHub: as-Lody) publicou o **VirtualApp** — open source, em Java. A pergunta que motivou: *“E se eu criasse um Android dentro do Android?”*

O VirtualApp intercepta chamadas Binder (o protocolo IPC do Android, criado por **Dianne Hackborn** baseado no OpenBinder da Be Inc. em 1996) usando Reflection para substituir os proxies locais por versoes proprias.

Em 2016, a **LBE Tech** lancou o **Parallel Space** baseado na mesma tecnica — 50 milhoes de downloads em meses. Bancos viram isso em fraudes a partir de 2017: o clone framework interceptava camera, GPS e verificacoes de seguranca antes do app financeiro acessar.

Camada 3: UID Offset — A Deteccao Mais Elegante

```
1 val userId = uidProvider() / 100_000  
2 if (userId != 0) {  
3     signals += DetectionSignal(  
4         SignalCategory.CLONE_APP,  
5         "App running under user ID $userId (non-primary user)",  
6         Severity.HIGH  
7     )  
8 }
```

Historia: Android Multi-User e UIDs (2012)

Ken Thompson e Dennis Ritchie criaram o sistema de UIDs no Unix em 1969. No Android 4.2 (2012), a Google adicionou suporte a multiplos usuarios para tablets compartilhados por familias. A decisao de engenharia: alocar UIDs em blocos de 100.000 por usuario.

Usuario	Range de UIDs
0 (primario)	10.000 – 99.999
1 (secundario)	110.000 – 199.999
2	210.000 – 299.999

O clone app precisa de usuario secundario. Nao ha como escapar. A decisao de engenharia de 2012 para familias em tablets virou, 12 anos depois, a deteccao mais confiavel contra clone apps.

```
1 // Device limpo:      myUid() = 10.153 / 100.000 = 0 -> sem
    sinal
2 // Dentro do clone: myUid() = 110.153 / 100.000 = 1 -> HIGH
```

Camada 5: /proc/self/maps e .so baked-in

```
1 File("/proc/self/maps").forEachLine { line ->
2     if (line.contains("io.va", ignoreCase = true)) {
3         signals += DetectionSignal(...)
4     }
5 }
```

Historia: ELF, shared libraries e paths baked-in (1988)

O formato **ELF** (Executable and Linkable Format) foi introduzido no Unix System V em 1988. Shared libraries (.so) tem seus paths de origem gravados pelo linker dentro do proprio binario compilado — o **rpath**.

Um atacante pode renomear o APK do VirtualApp com **apktool**. Mas o **libva.so** dentro do APK e um binario compilado — o **apktool** nao toca nele. O path **com.lody.virtual** continua baked-in no binario.

O **/proc/self/maps** — criado por Tom Killian no Bell Labs em 1984 — expoe quais .so estao carregadas na memoria. O kernel nao mente.

Injecao de Dependencias para Testabilidade

Historia: Dependency Injection e TDD (Michael Feathers, 2004)

Michael Feathers formalizou em 2004 no livro *“Working Effectively with Legacy Code”*: qualquer codigo que nao pode ser testado sem o ambiente real e legacy code, independente de quando foi escrito.

A solucao: substituir chamadas ao sistema por parametros injetaveis. No Kotlin moderno, uma lambda com valor default resolve sem frameworks:

```
1 // Producao: usa sistema real (nao passa nada)
2 val analyzer = CloneAnalyzer()
3
4 // Teste: simula UID 110.000 (usuario secundario)
5 val analyzer = CloneAnalyzer(uidProvider = { 110_042 })
```



VirtualizationAnalyzer.kt — Detectando Hooking

```
1 override fun analyze(context: Context): List<DetectionSignal> =
2     checkHookFiles() +
3     checkProcMaps() +
4     checkXposedClassLoader() +
5     checkFrida()
```

checkHookFiles() — Artefatos de Cada Framework

Historia: Xposed Framework (rovo89, Alemanha, 2012)

rovo89 era um desenvolvedor alemão frustrado com a impossibilidade de modificar apps do sistema Android sem recompilar o Android inteiro. A solução: substituir `/system/bin/app_process` — o programa que inicia o Zygote e é pai de todo app Android — por uma versão que injetava `XposedBridge.jar` no classpath de todo processo.

Historia: LSPosed (comunidade, 2020)

O Xposed modificava `/system` diretamente, quebrando o **dm-verity** (verificação criptográfica de integridade da Google). O LSPosed usou Magisk como base — monta um overlay virtual sem tocar no sistema real.

Historia: Cydia Substrate (Jay “saurik” Freeman, EUA, 2008)

Jay Freeman criou o Cydia em 2008 — loja alternativa para iPhone com jailbreak. O Cydia Substrate usava `LD_PRELOAD` (feature do linker Unix) para forçar o carregamento de sua biblioteca antes de qualquer outra.

Historia: Magisk (topjohnwu / Wang Guang, Taiwan, 2016)

Estudante taiwanês criou o Magisk: **systemless root** — monta `tmpfs` (filesystem em RAM) sobre `/system`. O Android vê modificações. O sistema real está intacto. O `dm-verity` fica feliz.

checkXposedClassLoader() — O Classpath como Detector

```
1 try {
2     Class.forName("de.robv.android.xposed.XposedBridge")
3     signals += DetectionSignal(..., Severity.HIGH) //
4         encontrou = Xposed ativo
5 } catch (_: ClassNotFoundException) { /* device limpo */ }
```

Historia: ClassLoader e Lazy Loading (James Gosling, 1995)

James Gosling projetou o Java em 1995 com **lazy loading** — classes só existem quando pedidas. O `ClassLoader` busca no classpath quando `Class.forName()` é chamado.

O Xposed injeta `XposedBridge.jar` no classpath do **Zygote**. Todo processo fi-

lho herda o classpath do pai. Logo, `de.robv.android.xposed.XposedBridge` fica acessível em qualquer app.

Se `Class.forName()` não lança `ClassNotFoundException`, o JAR do Xposed está no classpath — o framework está ativo.

checkFrida() — A Detecção Mais Sofisticada

Historia: Frida (Ole Andre V. Ravnas, Noruega, 2014)

Ole Andre queria algo que o GDB (criado por Richard Stallman em 1986) não oferecia: injetar JavaScript num processo em execução, sem pausar, interativamente. Criou o Frida em 2014.

Arquitetura cliente-servidor: `frida-server` roda como root no device, usa `ptrace()` — chamada de sistema Unix criada em 1979 no Unix V7 pelos engenheiros da Bell Labs — para injetar `frida-agent.so` no processo alvo via o dynamic linker.

```
1 // Detecção 1: probe de socket (150ms timeout)
2 Socket().use { socket ->
3     socket.connect(InetSocketAddress("127.0.0.1", 27042), 150)
4     signals += DetectionSignal(..., Severity.HIGH)
5 }
6
7 // Detecção 2: /proc/net/tcp fallback
8 if (signals.none { it.description.contains("Frida server") }) {
9     signals += checkFridaTcpTable("/proc/net/tcp")
10    signals += checkFridaTcpTable("/proc/net/tcp6")
11 }
```

Bug encontrado e corrigido: porta Frida

Durante o desenvolvimento foi encontrado um bug crítico:

```
1 // Código com bug:
2 val fridaPortHex = "699A" // = 27034 decimal -- ERRADO
3
4 // Código correto:
5 val fridaPortHex = "69A2" // = 27042 decimal -- CORRETO
```

`hex(27042) = 0x69A2`. O código anterior procurava a porta 27034 no `/proc/net/tcp`. O Frida escuta em 27042. O fallback inteiro era ineficaz — silenciosamente, sem erro.

Os testes passavam porque testavam a lógica do parser com linhas sintéticas, não a conversão do número da porta.

O formato do `/proc/net/tcp`

Historia: Big-Endian vs Little-Endian (1978 vs 1974)

O arquivo `/proc/net/tcp` mistura duas convenções históricas:

```
1 local_address = "0100007F:69A2"  
2             ^^^      ^^^  
3             little-endian  big-endian  
4             (Intel 1978)   (TCP/IP 1974)
```

O IP 0100007F usa little-endian porque o kernel Linux roda em processadores Intel x86, que desde 1978 armazenam bytes do menos significativo primeiro. A porta 69A2 usa big-endian (network byte order) porque o protocolo TCP foi definido por **Vint Cerf** e **Bob Kahn** em 1974 com bytes do mais significativo primeiro.

O estado 0A = 10 decimal = TCP_LISTEN — a maquina de estados TCP foi formalizada por Cerf e Kahn no mesmo paper de 1974.

Danny Cohen da USC cunhou os termos “big-endian” e “little-endian” em 1980 no paper “*On Holy Wars and a Plea for Peace*”, referenciando o livro *As Viagens de Gulliver* de Jonathan Swift.

RootDetector.kt — Detectando Root

```
1 class RootDetector(  
2     private val fileProbe: (String) -> Boolean = {  
3         File(it).exists() },  
4     private val writableProbe: (String) -> Boolean =  
5         { File(it).run { exists() && canWrite() } },  
6     private val packageChecker: (Context, String) -> Boolean =  
7         { ctx, pkg ->  
8             runCatching { ctx.packageManager.getPackageInfo(pkg,  
9                 0); true }  
10            .getOrElse(false)  
11        },  
12 ) : EnvironmentAnalyzer
```

Historia: Root no Unix e Android (1969 – hoje)

Ken Thompson e Dennis Ritchie criaram o usuario **root** (UID 0) no Unix em 1969 — acesso irrestrito, sem limitacoes.

No Android, root nao existe por padrao. A Google removeu deliberadamente. Mas a comunidade criou ferramentas:

SuperSU (Chainfire, Holanda, 2012) — o padrao-ouro por anos. Mostrava um popup pedindo permissao cada vez que um app tentava elevar privilegios. Em 2017, Chainfire vendeu para uma empresa chinesa (CCMT). A comunidade migrou.

Magisk (topjohnwu / Wang Guang, Taiwan, 2016) — systemless root. O Android tem **dm-verity**: verificacao criptografica de cada bloco da particao **/system** no boot. Root tradicional quebrava o dm-verity. Magisk monta **tmpfs** sobre **/system** sem modificar o original.

KingRoot, Kingo Root (China, 2013-2014) — populares no mercado asiatico por rootear sem desbloquear o bootloader, explorando vulnerabilidades do kernel.

Camada 1: checkSuBinaries()

```
1 internal fun checkSuBinaries(): List<DetectionSignal> =  
2     DetectionSignatures.SU_PATHS.filter { fileProbe(it) }.map {  
3         path ->  
4             DetectionSignal(SignalCategory.ROOT, "Root binary  
5                 found: $path", HIGH)  
6     }
```

O binario **su** (switch user) e um dos mais antigos do Unix. Em Android de producao, nao existe. Ferramentas de root o instalam em: **/system/bin/su**, **/system/xbin/su**, **/sbin/su**, **/su/bin/su**.

Camada 3: checkDangerousProps()

```
1 val explanation = when (prop) {
2     "ro.debuggable" -> "ro.debuggable=1 habilita JDWP em
        qualquer processo..."
3     "ro.secure"      -> "ro.secure=0 concede shell root via ADB
        sem autenticacao..."
4     "ro.build.type"  -> "Build type 'eng' ou 'userdebug' indica
        ambiente nao-producao..."
5 }
```

Historia: JDWP e ro.debuggable

JDWP (Java Debug Wire Protocol) foi criado pela Sun Microsystems em 1998 como parte do Java Platform Debugger Architecture. Permite que qualquer processo seja depurado via rede — leitura de memoria, modificacao de variaveis em runtime.

`ro.debuggable=1` habilita JDWP em qualquer processo do sistema, incluindo apps com `android:debuggable=false`. Em builds de producao sempre e 0. Presente apenas em builds de engenharia ou dispositivos rootados.

`ro.build.type` tem tres valores: **user** (producao), **userdebug** (depuracao habilitada), **eng** (root automatico).

Camada 4: checkWritableSystemPaths()

```
1 internal fun checkWritableSystemPaths(): List<DetectionSignal> =
2     DetectionSignatures.WRITABLE_SYSTEM_PATHS.filter {
3         writableProbe(it) }.map { path ->
4         DetectionSignal(SignalCategory.ROOT,
5             "System path is writable -- indicates root mount:
                $path", HIGH)
6     }
```

Em todo device de producao, `/system` e montado como **read-only** pelo `init` no boot. O `dm-verity` garante isso criptograficamente. `File("/system").canWrite() == true` e impossivel em device limpo.

SignatureAnalyzer.kt — Verificando Integridade do APK

```
1 override fun analyze(context: Context): List<DetectionSignal> =  
2     checkCertificate(context) +  
3     checkInstallerSource(context) +  
4     checkPackageNameConsistency(context)
```

Historia: Criptografia de Chave Publica e X.509 (1976-1988)

Whitfield Diffie e **Martin Hellman** publicaram em 1976 “*New Directions in Cryptography*” — inventando a criptografia de chave publica. Voce tem duas chaves matematicamente ligadas: a privada (so voce tem) assina, a publica (todos podem ter) verifica. Impossivel forjar sem a privada.

Em 1988, a **ITU-T** publicou o **X.509** — formato padrao para certificados digitais. Um certificado X.509 contem: subject (quem e), issuer (quem emitiu), validade, chave publica e assinatura.

O Android usa X.509 para assinar APKs desde a versao 1.0.

Camada 1: Debug Key Detection

```
1 val cert = CertificateFactory.getInstance("X.509")  
2     .generateCertificate(ByteArrayInputStream(sig.toByteArray()))  
3     as X509Certificate  
4 val subject = cert.subjectDN.name  
5 if (subject.contains("Android Debug", ignoreCase = true)) {  
6     signals += DetectionSignal(..., Severity.MEDIUM)  
7 }
```

Quando voce instala o Android Studio, ele cria automaticamente um **debug keystore** com subject sempre igual a CN=Android Debug, O=Android, C=US. Todo desenvolvedor no mundo tem esse mesmo subject.

Quando um atacante usa apktool para modificar e reassinar um APK, ele usa sua propria debug key. O subject CN=Android Debug no APK de producao e o fingerprint mais obvio de um repackaging.

Camada 2: SHA-256 Hash Comparison

```
1 private fun computeHash(signature: Signature, algorithm:  
2     String): String =  
3     MessageDigest.getInstance(algorithm)  
4         .digest(signature.toByteArray())  
5         .joinToString("") { "%02X".format(it) }
```

Historia: SHA-256 (NSA, 2001)

SHA (Secure Hash Algorithm) foi desenvolvido pela **NSA** e publicado pelo **NIST** em 1993. O SHA-256, parte da familia SHA-2, saiu em 2001.

Propriedade fundamental: qualquer mudança no input, por menor que seja, muda completamente o output. Trocar um bit no APK muda o hash inteiramente.

`MessageDigest` foi introduzido no Java 1.1 em 1997 pela Sun Microsystems como parte do Java Cryptography Architecture.

A API Dividida: Android P (2018)

```
1 if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.P) {
2     // Android 9+ (2018): APK Signature Scheme v3, suporta
      rotacao de chaves
3     pm.getPackageInfo(packageName, GET_SIGNING_CERTIFICATES)
4         .signingInfo?.apkContentsSigners
5 } else {
6     @Suppress("DEPRECATION")
7     pm.getPackageInfo(packageName, GET_SIGNATURES).signatures
8 }
```

O Android 9 introduziu o APK Signature Scheme v3 com suporte a rotação de chaves. A API antiga não suportava múltiplos certificados históricos e foi depreciada.

DetectionSignatures.kt — O Catalogo Centralizado

```
1 internal object DetectionSignatures {
2     val KNOWN_CLONE_PACKAGES = listOf(
3         "com.lbe.parallel.intl",    // Parallel Space
4         "com.lody.virtual",        // VirtualApp
5         "me.weishu.exp",            // VirtualXposed
6         // ... 21 outros
7     )
8     val SU_PATHS = listOf("/system/bin/su", "/system/xbin/su",
9         ...)
10    val ROOT_PACKAGES = listOf("com.topjohnwu.magisk",
11        "eu.chainfire.supersu", ...)
12    val HOOK_PATHS =
13        listOf("/system/framework/XposedBridge.jar", ...)
14    val LEGITIMATE_INSTALLERS = setOf("com.android.vending",
15        ...)
16    // ...
17 }
```

Historia: Single Responsibility Principle (Robert C. Martin, 2000)

Robert C. Martin (Uncle Bob) formalizou o **Single Responsibility Principle** em 2000: cada modulo deve ter apenas uma razao para mudar.

Antes desse arquivo, cada analisador mantinha suas proprias listas de assinaturas. Quando um novo clone app aparecia, voce precisava atualizar varios arquivos. **DetectionSignatures** centraliza: so um arquivo muda quando a threat intelligence muda.

internal object garante que o catalogo e visivel em todo o modulo **:app** mas invisivel fora dele — os dados de seguranca nao vazam para dependencias externas.

AndroidManifest.xml — A Configuracao

```
1 <queries>
2     <package android:name="com.lbe.parallel.intl" />
3     <package android:name="com.lody.virtual" />
4     <!-- ... 22 outros clone apps -->
5 </queries>
```

Historia: Package Visibility Filtering no Android 11 (2020)

Antes do Android 11, qualquer app podia chamar `PackageManager.getInstalledPackages()` e ver todos os apps instalados.

O Google, preocupado com privacidade, introduziu o **Package Visibility Filtering** no Android 11 (API 30, 2020): apps são invisíveis ao `PackageManager` a menos que explicitamente declarados no `<queries>`.

Sem essa lista, no Android 11+, o `CloneAnalyzer` ficaria completamente cego para apps clone — `getPackageInfo("com.lbe.parallel.intl")` lançaria `NameNotFoundException` como se o app não existisse.

A alternativa seria `QUERY_ALL_PACKAGES`, mas o Google rejeita apps na Play Store que pedem isso sem justificativa forte.

Conceitos Transversais

O Padrao de Injecao de Dependencias

Todos os analisadores usam o mesmo padrao:

```
1 class RootDetector(  
2     private val fileProbe: (String) -> Boolean = {  
3         File(it).exists() },  
4     // ...  
5 )
```

Em producao: funcoes reais do sistema (nao passa nada, usa defaults). Nos testes: lambdas que simulam qualquer cenario sem precisar de device.

Esse padrao foi formalizado por **Michael Feathers** em 2004 e e a base para todos os testes unitarios do projeto (usando **MockK**).

runCatching e Result<T>

```
1 runCatching { parseTcpTable(File(path).readLines(), path) }  
2     .getOrElse(emptyList())
```

Em C (Dennis Ritchie, 1972), erros eram tratados por valor de retorno. Em C++ (Bjarne Stroustrup, 1983) e Java (James Gosling, 1995), vieram as excecoes com **try/catch**.

Linguagens funcionais dos anos 80 (Haskell, ML) propuseram tratar o erro como um **valor** — **Result<T>** do Kotlin encapsula o resultado como sucesso ou falha sem interromper o fluxo.

O Binder IPC do Android

Historia: OpenBinder e o protocolo de comunicacao do Android

Em 1996, a **Be Inc.** criou o **OpenBinder** para o BeOS — comunicacao entre processos via mensagens estruturadas. A Be Inc. faliu em 2001. **Dianne Hackborn**, que trabalhava la, levou a ideia para o Google em 2005 e reimplementou como **Binder IPC**.

Todo acesso ao sistema Android (PackageManager, LocationManager, etc.) passa pelo Binder. O Android coloca dentro de cada processo um proxy local: `IPackageManager.Stub.Proxy` — o nome segue o padrao **CORBA (1991)** e **Java RMI (1997)**: Stub (lado servidor, desempacota) e Proxy (lado cliente, empacota).

O VirtualApp substituiu esses proxies via Reflection para interceptar todas as chamadas de sistema — sem root, apenas com APIs publicas do Java.

Bug Encontrado e Corrigido

Durante o desenvolvimento foi identificado um bug critico no `VirtualizationAnalyzer.parseTcpTable`

```
1 // ANTES (com bug):
2 val fridaPortHex = "699A"    // 0x699A = 27034 decimal -- PORTA
   ERRADA
3
4 // DEPOIS (corrigido):
5 val fridaPortHex = "69A2"    // 0x69A2 = 27042 decimal -- PORTA
   CORRETA
```

Como o bug foi encontrado

```
1 >>> hex(27042)                # porta real do Frida
2 '0x69a2'
3 >>> int('699A', 16)           # o que estava no codigo
4 27034                         # ERRADO
5 >>> int('69A2', 16)           # o que deveria estar
6 27042                         # CORRETO
```

Por que os testes nao pegaram

Os testes do `parseTcpTable` testavam a **logica do parser** com linhas sinteticas (rows com "69A2" hardcoded nos dados de teste), nao a **conversao numerica** da constante `FRIDA_PORT` para hex.

O fallback inteiro era ineficaz em producao: procurava a porta 27034 enquanto o Frida escutava em 27042. Silenciosamente, sem execucao, sem aviso.

Impacto

Com o bug, um atacante usando `frida-server` com a porta padrao teria o probe de socket falhar (se bloqueado) E o fallback de `/proc/net/tcp` retornar falso negativo. Deteccao via TCP completamente cega.

Resumo: Decadas de Historia em 500 Linhas de Kotlin

Tecnica	Origem	Como usada no projeto
UIDs Unix	Bell Labs, 1969	detecta clone apps pelo UID offset
/proc filesystem	Bell Labs, 1984	le <code>maps</code> e <code>net/tcp</code>
ELF shared libs	Unix System V, 1988	paths baked-in nos <code>.so</code>
X.509 certificates	ITU-T, 1988	verifica assinatura do APK
Java Reflection	Sun, 1997	acessa APIs internas do Android
SHA-256	NSA/NIST, 2001	hash do certificado de assinatura
Binder IPC	Be Inc./Google, 1996-2005	base do VirtualApp que detectamos
<code>ptrace()</code>	Bell Labs, 1979	base do Frida que detectamos
big/little-endian	Intel/TCP, 1974-1978	parse do <code>/proc/net/tcp</code>
Android multi-user	Google, 2012	UID / 100.000 detecta clone
Kotlin coroutines	JetBrains, 2018	analísadores em paralelo
Bayesian probability	Thomas Bayes, 1763	<code>emulatorProbability</code>

*“A melhor detecção de segurança não inventa novos truques.
Ela usa as limitações estruturais que os atacantes não conseguem esconder.”*